SYSTEM DESIGN: ZERO TO INTERVIEW HERO

System Design Interview Cheat Sheet

How to think through any system design question: what to bring up, when to bring it up, what to say, and what it costs. Every rule and number here comes from the course. Numbers in examples are the course's own made-up assumptions, so say yours out loud the same way.

1 The four steps

1 Understand

`3 to 10 min`

Ask questions, set the scope, do the math. Don't draw yet.

2 High level

`10 to 15 min`

Draw only what this problem needs, follow one request, and agree on it.

3 Deep dive

`10 to 25 min`

Go deep on the hard part. Follow their hints. This takes the most time.

4 Wrap up

`3 to 5 min`

Conclude, say what could still break, what you'd watch, and what's next.

> **"What specific feature do we want to build? What are we focusing on?"**

> **"How many users are we serving? How fast do we expect to scale up?"**

> **"Are there other features connected to this one that we need to take into account?"**

> **"Would you like me to go through the API and the tables now, or should we keep going?"**

> **"I'd like to go deeper on [this part]. Does that sound good?"**

> **"I'm a little stuck here, could you give me a hint?"**

How to run it

- Never assume. If you really must, say it out loud and check it before you build on it.
- Treat your interviewer like a coworker. Think out loud and check in.
- Say why each box is there. Add a piece only when it solves a real problem.
- Hints sound like small questions: "How would you keep this fast?" Follow them, even if you planned something else.
- You won't finish a big design. Say so, and agree on what to go deep on. That one question gets you a hint for free.
- Watch your time. Don't spend 10 minutes on Redis against Memcached before you've said what the cache does.
- Stuck? Don't go quiet. Say where you are and what you're weighing.

What to talk about in each step

A map of where topics usually come up. The topic cards later in this sheet say when each one is worth bringing up at all.

Step	What usually comes up	Listen for
1. Understand	The feature and scope. Who uses it and how many. **The question that changes everything** (see below). How fresh, how fast, how sure. What matters most: accuracy, not losing data, speed or cost. Encryption and privacy, because they change the design. Then the math: requests a second, the peak, storage, bandwidth, and what each number decides.	Vague words like "fast" or "real time": ask which kind. Numbers you'll need later. Anything that removes a whole system ("skip suggestions", "the next day is fine").

Step	What usually comes up	Listen for
2. High level	The API: what goes in and what comes out. The main boxes and why: DNS, load balancer, stateless web servers, the database and why that kind, object storage and a CDN for big files, a cache if reads dominate, a queue if there's slow work or bursts, the kind of connection if the server talks first. Split it into its flows (writing and reading, upload and watch). Follow one request from start to finish.	"Is this enough, or is there anything else you're looking for?" Their reaction tells you where to go deep.
3. Deep dive	The hard part of this question. The data model and its keys. Sharding and the sharding key. Consistency and conflicts. Hot keys and celebrities. Fan-out. Delivery promises, retries and duplicates. Order. The special index (a trie, a geohash, a Bloom filter). The rate limiting algorithm. What happens when each box dies.	"What happens if...", "how would you keep this fast", "what about a celebrity". Those are hints, so follow them.
4. Wrap up	A short conclusion. Bottlenecks: what struggles first. Failures and error cases. 10 times the users. What you'd monitor, with specific numbers. Cost. Privacy if it hasn't come up. What you'd do with more time.	Never say your design is perfect. There's always something to improve.

The question that changes everything

Most questions have one answer that reshapes the whole design. Find it, and ask it first.

Ask	Why it changes everything
Do we keep the data after it's delivered, or does it live on the device?	Same users, same messages: in the course's chat math, storage went from about 100 PB to about 4 PB, around 25 times less, with no math needed to ask it.
Things that stay still, or things that move?	Still things can be organized ahead of time. Moving things make writing the main job.
Friends or followers? Newest first or ranked? Who you follow, or recommended?	Followers are lopsided, so one post can become millions of writes. A recommended feed barely needs fan-out at all.
When you say queue, a mailbox or a log?	A mailbox deletes what's read. A log keeps it for several readers, replay and order.
Is it encrypted end to end?	Anything that needs to read the data (search, previews, dedupe, distance checks) moves to the device or goes away.
How big can it be? How short? How fresh?	One size answer ruled out UUIDs. One length answer ruled out Snowflake IDs. "The next day is fine" removed live updates.
A product for people, or an API for other programs? Who pays?	Different customers compete for the same machines, and not all of them are equal.

2 Estimation playbook

How to estimate out loud

- Do it even if nobody asks: it's a sanity check. Round, because your reasoning matters more than your arithmetic.
- Write your assumptions down, so you and your interviewer can change one.
- Label units. Write 200 KB, not 200. Mixing up MB and GB can break a design.
- Design for the peak, not the average, and say what peak you assume.
- Say which numbers you made up. A guess can be wrong, as long as you said it's a guess.
- Connect every number to a decision. Numbers aren't for memorizing.

Doing it in your head

- Multiply the numbers in front, then add up the zeros. 10 million × 100 bytes: 7 zeros + 2 zeros = 9 zeros, so **1 GB**.
- 50 million × 2 MB: 50 × 2 = 100, and a million × a million is a trillion bytes. So **100 TB**. Dividing works the same way: cross out zeros.
- A day has **86,400** seconds. In your head, use **100,000**.
- Say: "I know it's really 1,024, but I'm going to use 1,000 to keep the math simple." Pick one and stay with it.
- Practice: requests a second, peak, storage, cache size, servers.

Formulas and rules of thumb

Estimate	Formula	Rule of thumb from the course	What it decides
Requests a second	Per day ÷ 86,400	7 billion a day ÷ 86,400 ≈ **81,000**. In your head: ÷ 100,000 gives 70,000. Close enough.	How many servers, how much load balancing
Peak	Average × a peak factor	Assume **2×** and say it. Bursts can be far worse: 5 million emails in 10 minutes is about **8,300 a second**, over 100× the average. Say "the average is lying".	Design for spikes; queues to absorb bursts
Reads vs writes	Writes × reads per write	Thousands of reads to 1 write means you can do the hard work ahead of time.	Cache, read replicas, precomputing
Sends vs deliveries	Sends × people reached	1 message to a group of 50 is about **50 deliveries**. 1 post to 300 followers is **300 writes**. One post isn't one post.	Fan-out on write or on read
Hidden multipliers	Actions × requests per action	A request for every typed letter turns 1 search into about **20** requests. A location report every 30 s is 2,880 a day per phone.	Debouncing, batching, longer intervals
Updates on a timer	Connected people ÷ interval	10 million ÷ 30 s ≈ **333,000 a second**. Every 5 s instead is **6×** the traffic.	Memory store, not a disk database
Storage	Items a day × size × how long you keep it × copies	Text is small and media explodes: 1 message in 10 with media was about **99%** of storage. If years are already in the item count, don't multiply by them again. Count **3 copies**.	Sharding, object storage and a CDN
Promised vs used	Users × quota vs what arrives	Free space promised isn't space used. People don't fill it, which is how anyone can afford to give it away.	Size for what arrives
Bandwidth	Bytes a second × 8	Networks count **bits**, files count **bytes**. 140 TB a day × 8 ≈ 1,100 terabits, ÷ 86,400 ≈ **13 gigabits a second**. Downloads come on top of uploads.	CDN, regions, whether speed is the hard part
Transfer time	Size × 8 ÷ link speed	1 GB at 20 megabits a second = 8,000 megabits ÷ 20 = **400 s**, almost 7 minutes.	Direct upload, small parts, resuming
Connections	People connected at once ÷ connections per server	**2 million** per server (a team tuning for it) gives a **best case**. Real servers do work and need spare room, so say it's more.	Number of stateful servers

Estimate	Formula	Rule of thumb from the course	What it decides
Work in progress	Starts a second × seconds each lasts	11,600 answers a second × 8 s each ≈ **93,000** being written at once. Twice as long means twice the machines.	Machines for slow work
Machines	Work a second ÷ work per machine	The per-machine number is a guess you'd measure. Do a second calculation (memory and CPU) and say which one decides.	Capacity, and which resource runs out first
Memory and cache size	Items × bytes each	500 IDs × 8 bytes × 10 million people = **40 GB**, which fits in memory. Whole posts would be 5 TB. Store IDs, not copies.	Whether it fits on one machine: replicas, not shards
Tokens	Messages a second × tokens per answer	1 token ≈ **4 characters**. 100 tokens ≈ 60 to 80 words. A book page ≈ **400** tokens. Requests stop meaning much when one is 50× another.	Estimate in tokens, then machines
Cost	GB sent × price per GB	7.5 million GB a day at 2 cents = **$150,000 a day**. A text costs about 1 cent, so careless retries cost real money.	Treating popular items differently, build or buy
Parts in a row	Multiply availabilities	99.9% × 99.9% ≈ **99.8%**: about 9 hours down a year becomes about 17.5. Every box that has to work can take you down.	Fewer required boxes, more copies
Copies side by side	Down only if all are down	2 copies where either can do the job only go down together.	Backups at every level
Cost of downtime	Rate × time down	At 81,000 a second, 1 minute is about **5 million** messages.	How many nines are worth paying for
Everyone with everyone	n × (n - 1)	100 servers checking each other: **9,900** messages a round. 1,000 servers: about 1 million. That's $O(n^2)$.	A server in the middle, gossip
Spreading a wave	Count ÷ seconds spread over	2 million reconnects spread over 1 minute ≈ **33,000 a second**, not 2 million in 1 second.	Jitter on retries and reconnects
Halving	How many cuts to reach 1	1,000 → **10** cuts, 1 million → **20**, 1 billion → **30**.	Indexes, trees, levels, lookups
How many fit	2^{bits}, $62^{characters}$	5 bits = 32, 10 = 1,024, 12 = **4,096**. 41 bits of milliseconds ≈ **69 years**. 62^7 ≈ **3.5 trillion**.	ID and code length
Rows vs requests	Points, not batches	Batching saves requests, not messages or points. The database still gets every row.	Size the store for items, not requests

Reference numbers

Unit	Really	Rounded
KB	1,024 bytes	A thousand
MB	1,024 KB	A million
GB	1,024 MB	A billion
TB	1,024 GB	A trillion
PB	1,024 TB	A thousand TB

The gap is about 2% at KB and about 10% at TB. Your guesses can be off by much more.

Operation	Takes	If memory took 1 s
Memory, CPU cache	Nanoseconds	1 second
Same data center, round trip	Hundreds of µs	
Disk seek	~10 ms	About a day
California to the Netherlands and back	~150 ms	About 17 days

A disk seek is about 100,000× slower than memory. The ratios matter, not the exact numbers.

Uptime	Nines	Down a year
99%	2	About 3.65 days
99.9%	3	Under 9 hours
99.99%	4	Under an hour
99.999%	5	A few minutes

An SLA sets what you get back, usually credits. It doesn't keep you up. A phone at 99% can't tell 4 nines from 5.

Number	Value	Number	Value
Seconds in a day	**86,400** (use 100,000)	Milliseconds in a year	About **31.5 billion**
Bits in a byte	**8**	Feels instant	About a tenth of a second, **100 ms**
Snowflake-style ID	**64 bits**, 8 bytes	UUID	**128 bits**, 16 bytes
Push notification payload	About **4 KB**	Basic text message	**160** characters, 70 with one emoji
Sequential disk writes vs random	About 600 MB/s vs 100 KB/s on 6 spinning drives (Kafka's docs)	S3 standard storage (Amazon's design)	**11 nines** durability, **4 nines** availability
Multipart upload	Recommended over **100 MB** (S3 docs)	Bloom filter at 1% wrong guesses	A bit under **10 bits** per item
1 degree of latitude	About **111 km**	Phone GPS, open sky	About **5 m**

Sizes the course guessed (say "my guess"): a text message 200 bytes (100 text + 100 metadata), a chat photo or video 200 KB, a phone photo 2 MB, a post or a small record 1 KB, a web page 500 KB, a short link row 100 bytes, an open connection 10 KB of memory, a location value 100 bytes.

3 Scaling and servers

Each card: when to bring it up, what to say, the trade-off to name, and the follow-ups. The tags show the step it usually belongs in.

Scaling up vs scaling out

HIGH LEVEL WRAP UP

BRING IT UP WHEN

- You start with one server.
- Traffic or data outgrows one machine.
- One crash would take everything down.
- They ask about 10 times the users.

SAY

- **Up** (vertical): add CPU or RAM. Simple, no code changes. Use it when you're small.
- **Out** (horizontal): add servers to a pool. Keeps growing and survives a crash. Only this removes the single point of failure.
- Say what you'd start with, when it stops working, and what comes next.

Trade-off: up has a hard limit and one machine is still one failure; out means several machines to manage, and servers must be stateless.

10 times the users?	More web servers, more replicas, a bigger cache, and shards when the data gets too big. You didn't build it on day one, but you know when you would.
Why not one huge machine?	Maybe for a while. But there's a limit, it's still a single point of failure, and the number keeps growing.

Load balancer and health checks

HIGH LEVEL

BRING IT UP WHEN

- You draw more than one web server.
- Anything has to survive a crash.

SAY

- It spreads requests over a pool, and the address users connect to never changes.
- Health checks: a small request on a schedule. A server that stops answering stops getting traffic.
- One balancer is its own single point of failure, so run 2, or let the cloud provider handle it.
- When jobs vary a lot in size (like AI answers), pick the least busy machine, not round robin.

Trade-off: failover isn't instant, so some users see errors for a moment. Keep spare room: 2 servers at 80%, one dies, and the other gets 160%.

What if it dies?	Run 2. Don't just draw the box and move on.
How does it know a server is down?	Health checks. And never say failover is instant.

Stateless vs stateful servers

HIGH LEVEL DEEP DIVE

BRING IT UP WHEN

- Sessions or logins exist.
- You want auto scaling.
- The design holds open connections for a long time (chat, live location, notifications to devices).

SAY

- Keep the web tier stateless: sessions go in a shared data store, so any server can help anyone.
- Sticky sessions work, but adding or removing servers is harder and a crash loses sessions.
- An open connection lives on one machine, so those servers are stateful. Draw 2 sides: a normal side, and a connection side.
- Service discovery (a list like ZooKeeper) picks a server near the user and not too busy. A "front desk" in memory records who's on which server, with an expiry that each server keeps renewing.

Trade-off: stateful servers are hard to restart. Stop sending new people, move clients off a few at a time, and wait for it to drain. A crash drops everyone on it at once.

How did server 1 know where to send it?	It asked the front desk. If you can't say how it knew, your interviewer will ask.
A connection server dies?	Its clients reconnect at once and can knock over the next server. Random delays spread the wave: 2 million over 1 minute is about 33,000 a second.

Multiple data centers and regions

DEEP DIVE WRAP UP

BRING IT UP WHEN

- Users really are around the world. Ask how many, and where, first.
- Losing a whole building isn't acceptable.
- The time budget is smaller than a trip across the world (about 150 ms).
- Rules say data has to stay inside a country.

SAY

- GeoDNS sends people to the nearest region.
- Replicate data between them, and put copies of critical data in different buildings.
- Automatic deployment keeps every location the same.
- Shared counters (like a rate limiter) can run near users at the edge and sync in the background.

Trade-off: complicated, expensive and hard to keep in sync. Copies across the country are much slower than inside one building, and data that exists only in the region that went down is missing.

Red flag	Data centers in 3 countries in your first 5 minutes. Add it when you need it.

The API

HIGH LEVEL

BRING IT UP WHEN

- Step 2, as the first box: what goes in and what comes out is enough.
- For a single feature, a quick API and table sketch helps. For a whole store, it's too much detail this early.

SAY

- POST creates something, GET reads it.
- Login token in the Authorization header, never in the web address. Addresses end up in logs and history, and HTTPS doesn't fix that.
- Sensitive inputs like a location go in a POST body, so they stay out of server logs.
- Scrolling: a **cursor** ("20 after this ID"), because page numbers shift when new items land on top.

STATUS CODES THE COURSE USES

2xx, 3xx, 4xx, 5xx	It worked; go somewhere else; a problem with your request; a problem on our side.
301 vs 302	301 moved for good: browsers may remember it, so less load but you miss clicks. 302 for now: you see every click and can change the target.
101, 304	101 switching protocols (a WebSocket starting). 304 not modified, with no page attached.
404, 410, 429	Not found. Gone, so stop (a dead device token). Too many requests, with Retry-After.
503, 529	Overloaded, and you did nothing wrong. Different from your own 429.

4 Databases and storage

Choosing a database

HIGH LEVEL

BRING IT UP WHEN

- Always, in step 2, once you know the data and how it's used.
- Ask: what do we read most, by what key, and do several writes have to happen together?

SAY

- Most developers should start relational. Joins answer questions you didn't plan for.
- Relational when writes must happen together (a new version and "this is now the newest"). That's the real reason, not the label: some NoSQL offers transactions now.
- Go non-relational for: super low latency, unstructured data or no relations, only saving and loading whole values, or massive amounts of data.
- It isn't all or nothing: orders relational, the cart in a key-value store.

Kind	Good at	Pick it when
Key-value	Put and get by exact key, like a coat check	Only ever looked up by the key: a cart, a short link, precomputed answers. It can't find a coat by its color.
Document	Each item a document, usually like JSON	Whole items with their own shape
Wide-column	Huge data; a range of sorted rows inside one key	Write-heavy data that rarely changes, read as "the latest 50 in this chat". One part of the key picks the folder, the other sorts inside it.
Graph	Connections between things	Keeping who's connected to whom. For "who follows whom", a 2-column table with an index on each does the job.

Trade-off: non-relational generally gives up joins. Never say "NoSQL because it scales". Tie it to the data and how it's used.

Why not key-value for chat?	You need a range inside a key, not one whole value.
ACID?	Atomic (all or nothing), Consistent (rules hold), Isolated (no tripping over each other), Durable (saved stays saved). The C in ACID isn't the C in CAP.

Replication

HIGH LEVEL DEEP DIVE

BRING IT UP WHEN

- Losing the database would lose everything.
- Reads far outnumber writes.
- "What if the database goes down?"

SAY

- All writes go to one primary, the single source of truth. Reads spread over replicas, usually more replicas than primaries.
- A replica dies: reads go to the others, or to the primary for a while. The primary dies: a replica is promoted.
- Read-heavy and the data fits on one machine? The answer is replicas, not shards.

Trade-off: copies take time. A replica can be behind, so read anything that must be newest from the primary. A promoted replica may miss the last writes. To lose nothing, wait for a replica's "got it" before saying done, which costs a little time on every write.

Is failover instant?	No. It takes time, and promotion is very complicated in production.

Sharding and the sharding key

DEEP DIVE

BRING IT UP WHEN

- The data doesn't fit one machine, or the writes are too much for one machine.
- Do the math first. If the index fits in memory and writes are low, you don't need it, and saying so is a strong answer.

SAY

- The key is the most important decision: it sends each query straight to one shard and spreads data evenly.
- Key by how you read: by user (all of one person's posts together), by conversation, by shared folder (so a transaction stays on one shard), by ID if you only look things up by ID.
- The key can protect privacy: points stored by road segment and hour, not by person, means nobody's movement file exists.

Trade-off: resharding moves data, a celebrity on one shard overwhelms it, joins across shards are hard (denormalize), and transactions across shards stop being easy.

Shard by country?	One shard could get almost everyone. Same with the first letter of a word, or by city (Tokyo next to the ocean).
Adding a shard?	The remainder rule moves most keys. That's what consistent hashing is for.

Consistent hashing

DEEP DIVE

BRING IT UP WHEN

- Data or work is split across servers, and servers get added or removed often: caches, sharded databases, a crawler's websites, pub-sub channels.
- If the number of servers almost never changes, the remainder rule is fine and simpler.

SAY

- Hash ÷ number of servers breaks when the number changes: 3 to 4 servers moves about **75%** of keys, 10 to 11 about **91%**.
- The ring: put servers and keys on a circle, walk clockwise to the first server. Really a sorted list and a search. Only about **1 in N** keys move.
- **Virtual nodes:** each server takes many spots, so each gets close to a fair share. A bigger machine can take more spots.
- Copies go on the next N **different** machines clockwise.

Trade-off: more spots means a longer list on every server. The ring says where a key should be. It doesn't move the data, and it doesn't fix one very popular key.

A server dies?	Only its keys move, to the next server. For a database, you need replicas or that data is gone.
The celebrity problem?	Not solved: one key is still one key. Cache it, split it across keys, or give it its own server.
Is a virtual node a replica?	No. Nothing is copied. It's one machine showing up on the ring many times.

Consistency and CAP

STEP 1 DEEP DIVE

BRING IT UP WHEN

- There are copies, and the network can break.
- Money, stock or the last item, next to carts, likes and profile pictures.
- "Is eventual consistency okay?"

SAY

- Over a network, partitions aren't optional. The real choice is what you do during one: refuse and stay correct, or answer and stay available. When the network is fine, you get both.
- Choose per kind of data: the last copy of a book needs one right answer; a cart can be fixed later.
- Say what the user would actually see.

Promise	Means	Good for
Strong	After a change, nobody sees the old one again. Sometimes you get no answer.	The last copy of a book
Weak	New or old, with no promise when that changes	A live view count
Eventual	If changes stop, all copies end up the same, with no exact time. Can leave 2 different copies.	A profile picture, a cart, online status

"Pick 2 of 3"	Say partitions aren't optional, so the choice is only during a failure.
"Banks are CP"?	Careful: the same store gives different answers for different data.
Strong across devices?	Impossible at the edges. Make one place the record of the latest version: strong in the middle, eventual around the edges.

Quorum, conflicts and repair

DEEP DIVE

BRING IT UP WHEN

- Copies with no leader: how many have to agree?
- 2 writes to the same thing at once.
- A server is down for a while, or gone for good.

SAY

- N copies; a write waits for W, a read for R. If **W + R > N**, they overlap. N 3, W 2, R 2 is the common start. Every copy still gets the write; W is only how long you wait.
- You wait for the slowest one you're waiting for.
- Conflicts: **last write wins** (clocks disagree and an update can be lost), **vector clocks** (keep both, the app merges, deleted items can come back), or **one place decides** with a version check, and the loser is kept as a conflicted copy.

With N 3	Gives	Think of
W 3, R 1	Fast reads	Rarely changed descriptions
W 1, R 3	Fast writes, slower reads	
W 2, R 2	One slow server can't hold you up	A cart
W 1, R 1	Fastest, can be stale; a write can be lost if that one server dies	A view counter

How do you know a server's down?	Gossip: each server raises its own heartbeat counter, shares its table with a few random servers, everyone keeps the highest numbers. A number that stops rising means down. No single checker to fail.
Down for 10 minutes?	Sloppy quorum and hinted handoff: the next healthy server takes the write with a note and hands it back. W + R > N no longer guarantees overlap.
Copies drifted for a long time?	Merkle trees: compare fingerprints from the top down and copy only what differs.

Bytes vs facts: object storage and the database

HIGH LEVEL

BRING IT UP WHEN

- Photos, videos, files: anything big.
- Uploads that could hold a server for minutes.
- Old data nobody reads often.

SAY

- **Bytes** (huge, almost never change) go in object storage. Treat it as a black box unless they ask you to design it. **Facts** (names, owners, which version is newest: tiny, change often, must be right) go in a database.
- Two roads: the client uploads straight to storage with a **pre-signed URL** (one file, limited time), and sends the server a small note. A status column goes from uploading to processing to ready.
- Big files go in small parts. After a drop, ask what arrived and resend only what's missing.
- A queue and workers process it afterwards. A CDN serves it.
- Move cold data to cheaper storage, but think about who needs it in a hurry.

Trade-off: a pre-signed URL is a secret in an address, but it does one small thing and expires. Durability (not lost) isn't availability (reachable now).

Why not through web servers?	The server holds on to a slow phone the whole time, mostly waiting.
Files that change?	Cut into blocks with fingerprints and send only what's missing. That one question does delta sync, dedupe and resuming. Nothing shows until a commit.

Indexes and data structures: let the question pick

DEEP DIVE

BRING IT UP WHEN

- The main question isn't "get by exact key": prefixes, ranges, nearby, "have I seen this", the top few.
- Start with a plain table and an index, then say exactly when it breaks.

SAY

- "What do I need to ask this data?" The structure picks itself, then say what it costs.
- An index makes a lookup O(log n), about 24 looks in 10 million rows. LIKE 'du%' can use it; a % on the left can't.
- A sorted list has one direction. The map has neighbors in every direction.

Question	Structure	Cost
By exact name	Hash table, O(1)	No order at all
Have I seen this?	Set, or a Bloom filter	A set costs space. A filter says "definitely not" or "maybe", and a wrong maybe costs a wasted read, or a page you never visit
In order, or a range	Tree / sorted index, O(log n)	Extra work to stay balanced
Starts with	Trie, with the top answers saved on each node	One popular word is written once per letter
Only the top few	Heap of K	Each item only has to beat the smallest kept
Nearby	Geohash (a sortable text column) or a quadtree (in memory, adapts to crowds)	Close places can share nothing: read 9 squares and measure real distance
Who's affected by X?	A list per X, like a book's index	Has to be kept up to date
Connections	Graph (dots and arrows)	

Why not index latitude?	A latitude range is a ring around the whole planet.
O(1) means instant?	No, it means it doesn't get worse. A lookup on another machine is O(1) and still a network trip.

Logs and append-only storage

DEEP DIVE

BRING IT UP WHEN

- Writes are heavy and data rarely changes.
- "Why is this so fast?" or "why not a database?"
- Gathering raw events to add up later.

SAY

- Only add to the end, never change what's written. Sequential writes are about the cheapest thing a disk does.
- Write path: commit log on disk, then memory, then a sorted file that never changes (an SSTable). The log protects a server that loses power; copies protect against a server gone for good.
- Read path: memory, then each file's Bloom filter newest first, then the file.
- Compaction merges files in the background. A delete is a tombstone.
- A log for events is cut into segments; old segments are deleted whole.

Trade-off: reading the newest version may check several files, and compaction is extra disk work. You can't change a line, only add.

5 Memory, caching and speed

When to bring up a cache

HIGH LEVEL DEEP DIVE

BRING IT UP WHEN

- The same slow question gets asked again and again.
- Reads far outnumber writes, and the data changes rarely.
- A slightly old answer is fine.
- The data never changes once made (a short link, a video piece): a cached answer can't go stale.
- A small number of items gets most of the traffic.

SAY

- Read-through: check the cache, else go to the database, save the answer, send it back.
- Five things: **when** (read often, changed rarely), **expiry** (not too short, not too long), **consistency** (an old price), **failures** (use several cache servers), **eviction** (something has to go).
- Pick the key carefully: a square, not an exact location; tiles and a time window, never exact coordinates.
- Cache things apart by how fast they change: a caption almost never, a like count every second.

Trade-off: stale data, and memory is volatile, so never keep the only copy there. If the cache dies, its traffic lands on the database.

Do we even need it?	Do the math. If the data fits and another replica would do the same, that's simpler. Measure query time, peak load and how often the same thing is asked.
Everything expires at once?	The thundering herd: every miss lands on the database. Add randomness to each expiry.
Deleted or blocked items?	Check again when you read, and drop them.
Permissions in a cache?	Keep those short, or a removed person keeps access for a while.

What lives in memory, and why

DEEP DIVE

BRING IT UP WHEN

- Every request needs it fast: counters, who's online, who's on which server.
- Huge numbers of writes for a value that's old in seconds.
- The whole index fits on one machine.

SAY

- Memory is the desk: fast and small, gone when the power goes. Disk is the filing cabinet across the room.
- Self-deleting values (a TTL) for anything that should vanish when updates stop: online status, a live position, who's on which server, a rate counter, a used event ID.
- Atomic steps in a shared store: INCR, save-only-if-empty (NX), or a short script that runs as one step.
- Sorted sets for lists ordered by score. Store IDs, not whole objects.

Trade-off: memory is expensive and volatile. Only put there what you can rebuild or can afford to lose. A lost feed list is rebuilt from the database.

Write every heartbeat to a database?	No. It belongs in memory. Only things that must outlast the connection (like "last seen") go to a database, and only when someone leaves.

CDN

HIGH LEVEL

BRING IT UP WHEN

- Static files: images, video, map tiles, downloads.
- Users far from your servers.
- The same file goes to many people.

SAY

- Requests go to the nearest CDN server. The first one fetches from the origin and keeps a copy until the TTL expires.
- Change a file: invalidate it through the provider, or version it (cover.jpeg?v=2).
- If the CDN is down, clients go to the origin.
- Streaming works with a CDN because it's small files over normal HTTP.

Trade-off: you pay for transfers, so don't cache rarely used files. When a few items get most views, keep only the popular ones there. A "private" header means a CDN won't keep it.

Doing the work ahead of time (precompute and fan-out)

DEEP DIVE

BRING IT UP WHEN

- Reads hugely outnumber writes.
- A little old is okay.
- Slowness is unacceptable (a suggestion that arrives late is useless).
- One write reaches many readers.

SAY

- **On read:** simple, always fresh, nothing wasted. Work grows with how many you follow, and you wait on the slowest of many lookups.
- **On write:** a ready list per reader, one read later. Work grows with how many follow you. Skip people who haven't come back in a month.
- **Hybrid:** fan out for most, fetch huge accounts on read, merge by ID. Pick a line, watch it, move it.
- Build big precomputed answers in a batch, beside the old version, check it, then flip a switch.

Trade-off: writes multiply, and some answers are never read. A bad idea when every change must show at once (a bank balance), or when most answers never get read.

Why not update live?	One change can touch every precomputed answer above it. If slightly old is fine, update in a batch.

Hot keys and the celebrity problem

DEEP DIVE

BRING IT UP WHEN

- One key, account, group, square or prefix is far busier than the rest.
- Any time you say "shard by" or "key by".

SAY

- Two different problems hide under one word. **One key read too much:** cache it with copies on several servers, split it into several keys, or give it its own server. **One account writes to too many:** don't fan out, fetch it on read.
- Pick your keys knowing the hot ones: everyone buying one book at midnight sends one key to one partition.
- A crowded area can show a count or a sample instead of everyone.

Trade-off: consistent hashing and virtual nodes even out the number of keys, not how busy they are. Limits help: friends lists have a cap, follower counts don't.

6 Communication and delivery

Message queues: mailbox or log

HIGH LEVEL DEEP DIVE

BRING IT UP WHEN

- Slow work nobody should wait for (an email after a purchase).
- Bursts bigger than what's behind you can take.
- A slow dependency could hold everything else up.
- Steps run at different speeds (downloading and parsing).
- Several teams need the same events.

SAY

- Two jobs: nobody waits for anybody, and a burst becomes a line. Producers and consumers scale on their own.
- Ask: mailbox or log? A **mailbox** deletes a message once it's handled: a list of jobs. A **log** keeps it, and every reader keeps its own bookmark: several teams, replay, order.
- One line per dependency, so one bad courier can't hold up the others.
- A worker borrows a message and says done only after the work, or it goes back in line.

Trade-off: the work happens later, not right away, and retries bring duplicates. Don't draw one when someone's waiting for the answer and there's no slow job.

Why a queue?	Don't only say scale. Say bursts, and that a slow dependency shouldn't hold up the whole system.
Order?	Only promised inside one partition. The key you choose decides what stays in order.
More workers than partitions?	The extras sit idle. The partition count is a ceiling, so choose it carefully up front.
A consumer dies?	The group rebalances (classically with a pause), and the next reader starts from the last committed bookmark, so some work may repeat.

When the server has to talk first

HIGH LEVEL

BRING IT UP WHEN

- Nobody asked for this message: chat, live position, sync, a streamed answer.
- Ask how fresh it has to be, and whether it goes both ways.

SAY

- Tell the story of why each one came next. Requirements decide.
- Login, profiles and uploads stay normal requests.
- When the app is closed, only Apple or Google can reach the phone. Push is a nudge, best effort, about 4 KB. Never the only way something arrives.

Option	How	Use it when	Cost
Polling	Ask every few seconds	Rarely	Almost all the work is saying no
Long polling	The server holds the request until there's news	One sentence, a few seconds late is fine	One way, a gap between requests
Server-sent events	One request, the server streams back	Only the server talks (streaming answers)	One way
WebSocket	An open connection, either side talks	Both ways, all the time	A server remembers you for hours: stateful
Push courier	Apple or Google wakes the phone	The app is closed	Best effort, small, can be dropped

The user is offline?	Save first, send a nudge, and the device catches up from its bookmark. Don't only say "send a push".
Why not WebSocket?	If it's one way and a few seconds late is fine, it's more than you need.

Delivery promises and duplicates

DEEP DIVE

BRING IT UP WHEN

- "Can you guarantee it arrives?" or "only once?"
- Anything retried: payments, orders, messages, notifications.

SAY

- You can promise not to lose it **before you hand it over**. Say "got it" only once it's safely stored. Past the handoff, it's out of your hands, so anything that must arrive needs a second path.
- **At most once:** mark done, then work. **At least once:** work, then mark done. Pick the mistake by what it costs.
- Exactly once is at least once, plus a receiver that can handle repeats.
- An ID made once by the caller (it can be a UUID). One atomic step: save this ID only if it isn't there yet (a uniqueness rule, or Redis NX), with an expiry.
- Free idempotency: fixed output names, blocks stored under their fingerprint, re-adding an ID to a sorted set.

Trade-off: save first can lose it; send first can send it twice. A password code: send first. A payment receipt: save first.

Why not check, then write?	2 workers both ask, both hear no, both send. It has to be one atomic step.
How sure is a write?	acks 0 (nobody waits), 1 (the leader has it), all (everyone in sync has it), plus a floor on how many in sync, or "all" quietly becomes 1.
When is "lost" fine?	When the newest value replaces the old one. Then: no saving, no retries, no acks, and ignore anything older than what you have.

Order and time

DEEP DIVE

BRING IT UP WHEN

- "Which came first?"
- Events for one customer, chat or account.
- Anyone suggests timestamps.

SAY

- Clocks on different servers don't agree, and a phone's clock can be changed.
- Ask what really needs order. It's usually per customer, chat or order, not everything. Global order means 1 partition on 1 machine.
- Per-scope order: a counter per chat with one atomic step. A gap (43, then 45) tells the client it's missing one.
- Time-sortable IDs are only roughly sorted. Fine for a feed, not for a conversation.

Trade-off: a key gives order and creates a possible hot key. Adding partitions changes what you divide by and breaks order for existing keys.

Retries, backoff, jitter and dead letters

DEEP DIVE FOLLOW-UP

BRING IT UP WHEN

- Any call to something that can fail.
- Many clients come back after a crash.

SAY

- **Backoff:** wait longer each time, like 1, 2, 4, 8, 16 seconds.
- **Jitter:** a random wait, so retries don't come back in waves.
- **A limit,** then a **dead letter queue** and an alert when it grows.
- Don't retry everything: 5xx, wait and retry. 429, slow down and follow Retry-After. Gone or corrupt, stop.

Trade-off: without these you get a retry storm that keeps something down. One bad message can block everything behind it in a partition; moving it aside can mix up that customer's order.

Batching, priority and backpressure

DEEP DIVE

BRING IT UP WHEN

- "Make it faster": ask, faster in what way?
- Urgent and bulk work share a pipe.
- More work than capacity.

SAY

- A round trip costs the same however small the message: send a truck, not a bike. Batching gives throughput and costs latency. Compress the whole batch.
- Separate urgent and bulk lines, so a marketing blast can't delay a password code. Give every message an expiry.
- Readers pull, so a slow reader falls behind instead of drowning.
- When full: a short line with a maximum wait, an honest error, retries with jitter, and a cheaper deal for work that can wait.

Trade-off: batching adds delay, and it saves requests, not items. Idle urgent workers are worth paying for.

7 Protection and operations

Rate limiting

HIGH LEVEL DEEP DIVE

BRING IT UP WHEN

- Anyone can create things, log in, scrape or scan.
- Expensive work, or a leaked key could run up a bill.
- One client could slow everyone else.

SAY

- Server side, as middleware or in an API gateway. The client side can't be trusted.
- Shared counters in memory (Redis), with one atomic step. Rules in a config file, so a bad limit is fixed without a deploy.
- Reject with a 429, headers and Retry-After, and words a person can read.
- For login, count per account too, not only per IP.
- When requests aren't equal, count their cost (tokens), not requests.

Algorithm	Good part	Cost	Fits
Fixed window	Simplest, least memory	A burst at the edge: up to 2× in a short time	Something simple
Sliding window log	Most accurate	A timestamp per request: lots of memory	Small strict limits (login)
Sliding window counter	Close, 2 numbers per client	An estimate	A general limit
Token bucket	Allows short bursts; size sets the burst, refill the average	2 numbers to tune; bursts still arrive	Real users in bursts (search)
Leaky bucket (queue)	A steady flow out	Requests wait	Sending to something with a fixed speed
Leaky bucket (meter)	Nobody waits	Like a token bucket	Say which leaky bucket you mean

The limiter goes down?	Fail open (stay up, nobody limited for a while) or fail closed (protected, but nobody gets in). It can differ per feature. A replica and a short timeout make it less likely.
Many servers?	Per-server counts let 3 servers allow 3×. Use one shared store, and atomic steps to stop the race.
Perfectly accurate?	No. Say how much off is okay, and why. Limits are targets, not guarantees.
Turning a new limit on?	Run it first in a mode that only writes down what it would block, then watch rejections per rule.

IDs

DEEP DIVE

BRING IT UP WHEN

- Rows across several databases need unique keys.
- IDs should sort by time, or must be short.
- Clients retry and you need to spot repeats.

ASK

- Unique everywhere? Numbers only? Going up with time? How big can it be? How many a second? How short? Must the same input give the same code?
- The size answer alone decides between a UUID and a 64-bit ID.

Option	Good	Breaks
Auto increment	Perfect on one database	2 shards hand out the same number
Skipping numbers	No talking needed	Not in time order; adding a database changes every one
Ticket server (2: odd and even)	Simple, cheap, a great answer for small systems	One machine is a single point of failure; a network trip each time
Ranges from a ticket server	Each server counts a block in memory; rare trips	Easy to guess; a crash loses a block
UUID	Make it anywhere, ask nobody	128 bits. Version 4 is random and indexes badly; version 7 sorts by time
Snowflake	64 bits, roughly sorted, asks nobody	Depends on the clock and unique machine numbers; long when written out
Hash of the input	Same input, same code; no shared service	Collisions come early, so a check on every write
A counter per scope	Exact order inside one chat	One atomic step per message; a busy scope is a hot key

SNOWFLAKE: 1 + 41 + 5 + 5 + 12 BITS

Sign bit always off, **41** bits of milliseconds (about **69 years**, so start the epoch near launch), **5** data center bits (32), **5** machine bits (32 each, 1,024 total), **12** sequence bits (4,096 per ms, about **4 million a second** per machine). Time comes first, so sorting by ID sorts by time, roughly. Trade bits between sections if needed.

BASE 62

10 digits + 26 small + 26 capital letters. $62^6 \approx 57$ billion, $62^7 \approx$ **3.5 trillion**. Convert by dividing by 62 and reading the remainders from the bottom up: 12,345 → 199 r 7 → 3 r 13 (d) → 0 r 3, so **3d7**. It's reversible to the number, unlike a hash. Remove look-alike characters (0, O, l, I) and about 58 are left.

Clock goes backward?	Refuse to make IDs until the clock passes the last time used. A pause beats a duplicate.
Sequence runs out?	Wait for the next millisecond. Don't say it fails.
2 machines share a number?	Silent duplicates. Claim machine numbers at startup.
Sent to a browser?	JavaScript can't keep whole numbers exact past about 9 quadrillion, so send big IDs as text.

Failure: what if this dies?

DEEP DIVE WRAP UP

BRING IT UP WHEN

- For every box you draw. Expect "what if this dies?" for each.

SAY

- Who takes over, and what users see meanwhile.
- Web server: the others carry on. Cache: traffic lands on the database. CDN: clients go to the origin. Database: promote a replica. Worker: its borrowed task goes back in line, and the task is safe to run twice. Connection server: clients reconnect with jitter.
- A long job saves its progress (a checkpoint), so a new machine picks up from the last save.
- Replace a dead server in the same spot, so nothing else moves.
- Set timeouts and size limits, so one silent dependency can't hang every thread.

Trade-off: a slow machine can look dead. Heartbeats and timeouts that are too short cause false alarms; too long and work sits stuck.

Monitoring and rolling out

WRAP UP

SAY

- Metrics tell you something's wrong, and logs help you find out what. Bring logs into one place.
- Name specific numbers: how long things take, how many fail, how often the cache has the answer; queue length and **age of the oldest message**; rejections per rate-limit rule; consumer lag; how long a change takes to reach another device; how often video stops to buffer.
- Watch whether the answers are actually good, not only whether it's up.

ROLLING OUT (FROM PUBLIC POSTMORTEMS)

- Roll out slowly, and watch what you're touching.
- Test at real size. Code that never runs isn't tested. Use feature flags.
- Keep a way in: do I need the broken thing to fix it?
- Randomized backoff when everything comes back at once.

Privacy and security

STEP 1 WRAP UP

BRING IT UP WHEN

- Messages, files, locations, searches: anything personal. Even if told to skip it, say one sentence first.
- Ask about encryption in step 1, because it changes the design.

SAY

- **In transit** stops someone on the wifi. **At rest** stops a stolen disk. We can still read both. **On the device** (end to end) stops us too: no server search, previews, dedupe or processing, and metadata (who, to whom, when) is still visible.
- Check permissions on the server on every request. Hiding a button stops nothing.
- Share links long and random, and expirable. A short link isn't a password.
- Don't keep what you don't need. Keep it out of logs. Make sharing opt in and expire.
- Removing the name isn't anonymous: 4 points of place and time picked out 95% of people.

Cost and build or buy

DEEP DIVE WRAP UP

BRING IT UP WHEN

- Bandwidth, storage or chip time numbers come out huge.
- A few items get most of the use.

SAY

- Start with the shape: treat the popular few differently from the long tail. Make extra versions only once something gets used.
- Move work that can wait to quiet hours, often at a lower price.
- Buy when you're small, build when the bill is bigger than the cost of building. Using existing storage and a CDN, with a cost, shows more judgment than building everything.

Trade-off: over-engineering costs the company money, literally and figuratively.

8 Questions you can expect anywhere

If they ask	You can say
What struggles first?	Find the bottleneck: usually whatever every cache miss ends up at. Say why.
What if this dies?	Who takes over and what users see. Failover takes a little time.
What about 10 times the users?	What you'd add, in order: web servers, replicas, a bigger cache, then shards when the data is too big.
Is eventual consistency okay?	Say what the user would see. An old picture for a moment is fine. 2 people paying for 1 book isn't.

If they ask	You can say
What about a celebrity?	Name which of the 2 problems it is: one key read too much, or one account writing to too many.
Can you guarantee delivery, or exactly once?	Up to the handoff. At least once, plus an ID that makes repeats harmless.
How do you make it faster?	Faster in what way? Time to the first result, or total throughput? Then cache, precompute or batch, and say the cost.
How do you make it cheaper?	The shape of the usage, the long tail, quiet hours, build or buy.
What would you monitor?	Specific numbers, like how long it takes, how many fail, cache hits, and the age of the oldest message in each queue.
What would you do with more time?	Name error cases and scale-up opportunities. There's always something.

9 Dos and don'ts

Do

- Ask good questions. It's the most important thing you can do.
- Do the math out loud, and connect each number to a decision.
- Say what you'd start with, when it stops working, and what you'd do next.
- Say what each choice costs, and tie it to the data and how it's used.
- Bring up failure, privacy and priority before they ask.
- Practice drawing with a pen, out loud, with a 45 minute timer.

Don't

- Make assumptions without checking them.
- Jump into a solution before you understand the requirements.
- Over-engineer, or ignore the trade-offs.
- Name a tool without saying why it fits this data.
- Overclaim: exactly once, perfectly sorted, perfectly accurate, five nines.
- Tell your interviewer that your design is perfect.

Remember: you grow, something breaks, and you fix it. Every piece in your design should be there to solve a real problem, and you should be able to say what it costs.